ML

Title: Predict the price of the Uber ride from a given pickup point to the agreed drop-off location.

Perform following tasks:

1. Pre-process the dataset.

2. Implement linear regression and random forest regression models.

3. Evaluate the models and compare their respective scores like R2, RMSE, etc.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error

# Load dataset
df = pd.read_csv("uber.csv")

# Preprocess dataset
df = df.dropna()
df = df.drop_duplicates()
df = df.drop(columns=['Unnamed: 0', 'key'], errors='ignore')
df = df[(df['fare_amount'] > 0) & (df['passenger_count'] > 0)]

df['pickup_datetime'] = pd.to_datetime(df['pickup_datetime'])
df['hour'] = df['pickup_datetime'].dt.hour
df['day'] = df['pickup_datetime'].dt.day
df['month'] = df['pickup_datetime'].dt.month

df = df.drop(columns=['pickup_datetime'])

df = df[
    df['pickup_longitude'].between(-180, 180) &
    df['dropoff_longitude'].between(-180, 180) &
    df['pickup_latitude'].between(-90, 90) &
    df['dropoff_latitude'].between(-90, 90)
]

# Select features and target
X = df.drop('fare_amount', axis=1)
y = df['fare_amount']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
```

```
)

# Linear Regression
lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

# Random Forest Regression
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)

# Evaluate models
print("Linear Regression")
print("R2 Score:", r2_score(y_test, y_pred_lr))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_lr)))

print("\nRandom Forest Regression")
print("R2 Score:", r2_score(y_test, y_pred_rf))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_rf)))
```

2)Title: Classify the email using the binary classification method. EmailSpam

detection has two states:

a) Normal State – Not Spam,

b) Abnormal State – Spam.

Use K-Nearest Neighbors and Support Vector Machine for classification.

Analyze theirperformance.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, classification_report

# 1. Load dataset
df = pd.read_csv("emails.csv", encoding="latin1")

# 2. Separate features and target
X = df.drop(columns=["Email No.", "Prediction"])
y = df["Prediction"]

# 3. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
# 4. Train and evaluate models
models = {
    "KNN": KNeighborsClassifier(n_neighbors=5, metric="cosine"),
    "SVM": LinearSVC(random_state=42)
}

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)

    print("\n", name)
    print("Accuracy:", round(accuracy_score(y_test, pred), 4))
    print(classification_report(
        y_test, pred,
        labels=[0, 1],
        target_names=["Not Spam", "Spam"],
        zero_division=0
    ))
```

3)Title: Given a bank customer, build a neural network-based classifier that can determine whether they

will leave or not in the next 6 months

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report

# 1. Load dataset
df = pd.read_csv("Churn_Modelling.csv")

# 2. Preprocess dataset
df = df.drop(columns=["RowNumber", "CustomerId", "Surname"])

df = pd.get_dummies(df, columns=["Geography", "Gender"], drop_first=True)

# 3. Separate features and target
X = df.drop(columns=["Exited"])
y = df["Exited"]

# 4. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 5. Scale features
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 6. Build and train neural network
model = MLPClassifier(
    hidden_layer_sizes=(16, 8),
    max_iter=300,
    random_state=42
)

model.fit(X_train, y_train)

# 7. Predict customer churn
y_pred = model.predict(X_test)

# 8. Evaluate model
print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))
print(classification_report(
    y_test, y_pred,
    target_names=["Will Stay", "Will Leave"],
    zero_division=0
))
```

4)Title: Implement K-Nearest Neighbors algorithm on diabetes.csv dataset. Compute confusion matrix,

accuracy, error rate, precision and recall on the given dataset.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score,
recall_score

# 1. Load dataset
df = pd.read_csv("diabetes.csv")

# 2. Separate features and target
X = df.drop(columns=["Outcome"])
y = df["Outcome"]

# 3. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
# 4. Scale data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 5. Apply KNN
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

# 6. Predict
y_pred = model.predict(X_test)

# 7. Evaluate model
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
error_rate = 1 - accuracy
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)

print("Confusion Matrix:\n", cm)
print("Accuracy:", round(accuracy, 4))
print("Error Rate:", round(error_rate, 4))
print("Precision:", round(precision, 4))
print("Recall:", round(recall, 4))
```

5) Title: Implement K-Means clustering/ hierarchical clustering on sales_data_sample.csv dataset. Determine the number of clusters using the elbow method.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering

# 1. Load dataset
df = pd.read_csv("sales_data_sample.csv", encoding="latin1")

# 2. Select numerical columns
X = df.select_dtypes(include=["number"]).fillna(0)

# 3. Normalize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 4. Elbow Method
wcss = []
```

```python
for k in range(1, 11):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X_scaled)
    wcss.append(model.inertia_)

# 5. Plot Elbow Graph
plt.plot(range(1, 11), wcss, marker="o")
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()

# 6. K-Means Clustering
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
df["KMeans_Cluster"] = kmeans.fit_predict(X_scaled)

print("K-Means Clustering:")
print(df["KMeans_Cluster"].head())
print(df["KMeans_Cluster"].value_counts().sort_index())

# 7. Hierarchical Clustering
hc = AgglomerativeClustering(n_clusters=4)
df["Hierarchical_Cluster"] = hc.fit_predict(X_scaled)

print("\nHierarchical Clustering:")
print(df["Hierarchical_Cluster"].head())
print(df["Hierarchical_Cluster"].value_counts().sort_index())
```